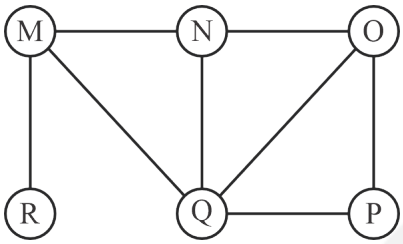
CS& DS

Artificial Intelligence

DPP: 1

Un-informed Search

Q1 The Breadth First Search (BFS) algorithm has been implemented using the queue data structure. Which one of the following is a possible order of visiting the nodes in the graph below?

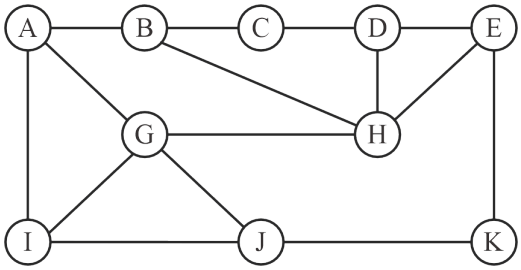


(A) MNOPQR
(B) NQMPOR
(C) QMNROP
(D) POQNMR

Q2 Statement 1: Given a graph G = (V, E) in which each vertex v ∈ V has an associated positive weight w(v), we can use linear programming to find the lower bound on the weight of the minimum-weight vertex cover.
Statement 2: The lower bound can be found by maximizing the following
$\sum_{v \in V}^{n} w(v)x(v)$
subject to
x(u) + x(v) > = 1 for each (u, v) ∈ V
x(v) $\leq$ = 1 for each v ∈ V
x(v) $\geq$ = 0 for each v ∈ V
In the light of the above statements, choose the most appropriate answer from the options given below :

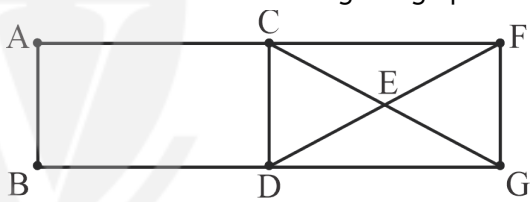
(A) Both statement I and Statement II are correct
(B) Both statement I and Statement II are incorrect
(C) Statement I is correct but Statement II is incorrect
(D) Statement I is incorrect and Statement II is correct

Q3 Consider the following graph.



Apply breadth first search traversal of the above graph. Which of the following traversal is possible if start vertex is G? (Assume lexicographic ordering)
(A) GAHJIBCEKD
(B) GAHJIBCDEK
(C) GAHIJBDEKC
(D) GAHIJBCEKD

Q4 Which sequence cannot be obtained while using BFS traversal for the below given graph?



I. ABCFGED
II. A B C G FED
III. ABDECFG
(A) Only I
(B) Only II
(C) Only II and III
(D) I, II, III

Q5 Breadth First Search (BFS) is started on a binary tree beginning from the root vertex. There is a vertex t at a distance 4 from the root. If t is the n-th vertex in this BFS traversal, then the maximum possible value of n is _____.

Q6 Let b be the branching factor of a search tree. If the optimal goal is reached after d actions from the initial state, in the worst case, how many

Android App | iOS App | PW Website

times will the initial state be expanded for iterative deepening depth-first search(IDDFS)and iterative Depending A* search(IDA*) ?

(A) IDDFS – d, IDA * – d

(B) IDDFS – d, IDA * – b^d

(C) IDDFS – b^d, IDA * – d

(D) IDDFS – b^d, IDA * – b^d

Q7 The time and space complexity of BFS is (For time and space complexity problems consider b as branching factor and d as depth of the search tree.)

(A) O(bd+1) and O(bd+1)

(B) O(b2) and O(d2)

(C) O(d2) and O(b2)

(D) O(d2) and O(d2)

Q8 Let G be a simple undirected graph. Let Td be a depth first search tree of G. Let Tb be a breadth first search tree of G. Consider the following statements.

I. No edge of G is a cross edge with respect to Td. (A cross edge in G is between two nodes neither of which is an ancestor of the other in Td).

II. For every edge (u, v) of G, if u is at depth i and v is at depth j in Tb, then |i – j| = 1.

Which of the statements above must necessarily be true ?

(A) I only

(B) II only

(C) Both I and II

(D) Neither I nor II

Q9 In a depth-first traversal of a graph G with 'n' vertices, 'k' edges are marked as tree edges. The number of connected components in G is:

(A) k

(B) k + 1

(C) n – k – 1

(D) n – k

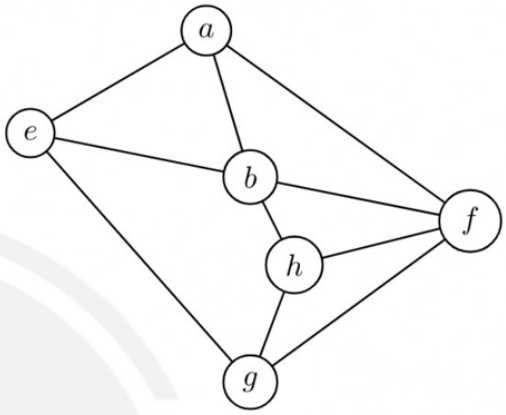
Q10 Consider a complete binary tree with seven nodes. Let A denote the set of the first three elements obtained by performing Breadth-First Search (BFS) starting from the root. Let B represent the set of the first three elements obtained by performing a Depth-First Search (DFS) starting from the source.

The value of |A – B| is _____ .

(A) 1 (B) 2

(C) 3 (D) 4

Q11 Consider the following graph:



Among the following sequences:

I → abeghf

II → abfehg

I → abfhge

I → afghbe

Which are the depth-first traversals of the above graph?

(A) I, II and IV only

(B) I and IV only

(C) II, III and IV only

(D) I, III and IV only

Q12 Let G = (V, E) be a simple undirected graph, and 's' be a particular vertex in it called the source. For, x ∈ V, let d(x) denote the shortest distance in G from 's' to 'x'. A breadth first search (BFS) is performed starting at 's'. Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T, then which one of the following CANNOT be the value of d(u) – d(v) ?

(A) 1 (B) 0

(C) 1 (D) 2

Android App | iOS App | PW Website

GATE

Answer Key

Q1	(D)	Q7	(A)
Q2	(C)	Q8	(A)
Q3	(D)	Q9	(D)
Q4	(D)	Q10	(A)
Q5	31	Q11	(D)
Q6	(A)	Q12	(D)

Android App | iOS App | PW Website

Hints & Solutions

Q1 Text Solution:

Concept:

Breadth first search involves search through a tree one level at a time. Traverse through one entire level of child node first, before moving to traverse through the grandchildren nodes.

Explanation:

(1) MNOPQR

In this, traversal starts from S but O is visited first before child node N and Q of M. So, incorrect.

(2) NQMPOR

It is also incorrect, P is traversed before O.

(3) QMNROP

Incorrect, because O must be traversed before R.

So, option 4) is correct.

Q2 Text Solution:

I'll break down the two statements and explain why they are both considered correct:

• Statement 1:

○ Given a graph G = (V, E) in which each vertex v ∈ V has an associated positive weight w(v), we can use linear programming to find the lower bound on the weight of the minimum-weight vertex cover.

• Explanation:

○ This statement is correct. Linear programming can be used to find the lower bound on the weight of the minimum-weight vertex cover by formulating it as an optimization problem.

• Statement 2:

○ The lower bound can be found by maximizing the following

○ $\sum_{v \in V}^{n} w(v)x(v)$

○ subject to

○ x(u) + x(v) >= 1 for each (u, v) ∈ E

○ x(v) $\leq$ = 1 for each v ∈ V

○ x(v) $\geq$ = 0 for each v ∈ V

• Explanation: This is incorrect. For the minimum weight vertex cover problem, the objective function should be minimized, not maximized. Also, the constraint x(u) + x(v) $\geq$= 1 should be for each edge (u, v) ∈ E (not V). With these corrections, the model will like this: It should be instead of

$\sum_{v \in V}^{n} w(v)x(v)$

• So, the corrected objective function should be:

$\sum_{v \in V}^{n} w(v)x(v)$.

Q3 Text Solution:

Breadth-first search is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root (or some arbitrary node of a graph) and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.

A queue is used to implement a breadth-first search.

Enque node G in the queue.

Queue: G

Mark it as visited. Dequeue G and enqueue its non-visited adjacent nodes.

Queue: A HIU

Mark A as visited. Dequeue A and enqueue its non-visited adjacent nodes.

Queue: HIJ B

Mark Has visited. Dequeue H and enqueue its non-visited adjacent nodes.

Queue: IJBDEK

Mark I as visited. Dequeue I and enqueue its non-visited adjacent nodes.

Queue: JBDEK

Mark J as visited. Dequeue J and enqueue its non-visited adjacent nodes.

Mark B as visited. Dequeue B and enqueue its non-visited adjacent nodes.

Queue: DEKC

Mark D as visited. Dequeue D and enqueue its non-visited adjacent nodes.

Queue: EK C

Mark E as visited. Dequeue E and enqueue its non-visited adjacent nodes.

Queue: K C

Android App | iOS App | PW Website

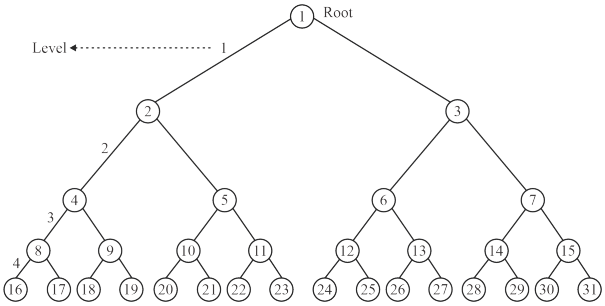
Mark K as visited. Dequeue K and enqueue its non-visited adjacent nodes.
Queue: C
Mark C as visited. Dequeue C and enqueue its non-visited adjacent nodes.
Queue: Empty queue
Hence traversal is GAHUBDEKC.

Q4 Text Solution:

Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures.
It starts at the tree root (or some arbitrary node of a graph, sometimes referred to as a 'search key'), and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.
Explanation:
Since in all the option traversal start with node A, Therefore A is the root node
Statement I: ABCFGED
A → B → C → FG (not allowed)
∵ D is at distance 2 and G is at distance 3 from root node
Statement II: ABCGFED
A → B → C → G (not allowed)
∵ D, E and Fare at distance 2 and G is at distance 3 from the root node
Statement III: A B D E C F G
A → B → D (not possible)
∵ C is at distance 1 and D is at distance 2 from the root node

Q5 Text Solution:

At a distance four the root, tree will be like :



In this tree, maximum possible value of n at a distance 4 from the root is 31.
Alternate solution:
A tree with height h has maximum number of nodes = $2^{h+1} - 1$.
Here it is given that t is the nth vertex at a distance 4. We have to find the maximum value of n.
Here height = 4
Maximum value of n = $2^{4+1} - 1 = 31$.

Q6 Text Solution:

As they have ask how many time root(start/initial stage) expand so in both they expand(iterate) it d+1 time so answer should be A.
If IDDFS storing state previously visited it may be less than it is 1 in IDDFS

Q7 Text Solution:

We consider a hypothetical state space where every state has b successors. The root of the search tree generates b nodes at the first level, each of which generates b more nodes, for a total of b2 at the second level. Each of these generates b more nodes, yielding b3 nodes at the third level, and so on. Now suppose that the solution is at depth d. In the worst case, we would expand all but the last node at level d (since the goal itself is not expanded), generating bd+1- b nodes at level d+1.

Q8 Text Solution:

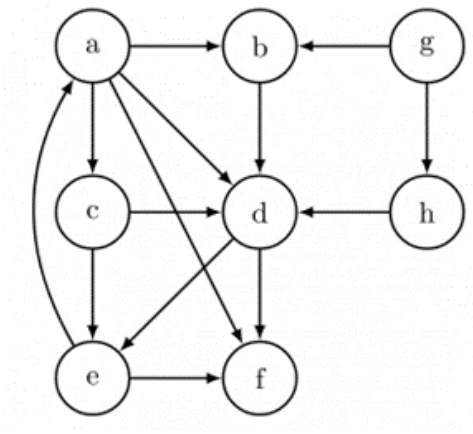
I. Since the graph is simple and undirected, according to DFS, each edge in Td is either Tree Edge or Back Edge, hence there are no cross edges. (Hence I is True)
(If you do not agree just try it with taking examples. You can't draw)

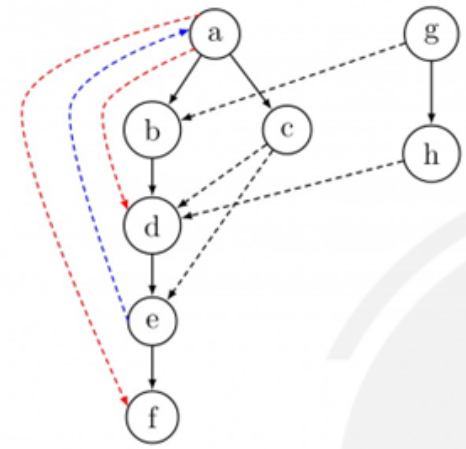
II. Just draw a triangle SAB. Source is S. Vertex A and B are at same level hence distance 1. So, here |i – j| = 0. (Hence II is False)

Hence answer is (A).
Anyway, II is simple to understand from the above explanation.

Q9 Text Solution:

Android App | iOS App | PW Website





Tree edge

Cross edge

Forward edge

Back edge

Tree edges are those edges which appear in the final DFS forest. For example in case of connected graph (i.e. resulting DFS forest containing only one tree), if we run DFS over it, edges belonging to the resulting DFS tree are called tree edges.

Let us assume the graph has 'x' number of connected (or strongly connected in case of a directed graph) components. And assume 1st component has 'k1' tree edges, 2nd component has 'k2' tree edges and xth component has 'kx' tree edges.

Such that (k1 + k2 + …… + kx = k i.e., total)

Or in other way we can imagine like, the final DFS forest has 'x' trees and those trees are having k1, k2, k3, ….., kx edges respectively.

Now we know that a tree having kx edges contains kx + 1 nodes. Similarly a tree having k1 edges contains k1 + 1 nodes, etc. and so on.

So, summation of nodes in each tree = n.

Correct Answer:

(k1 + 1) + (k2 + 1) + (k3 + 1) + …… + (kx + 1) = n

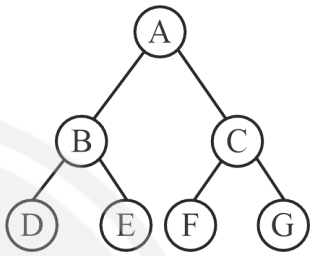
(k1 + k2 + k3 + ….. + kx) + x = n

x = n – k

Therefore, correct answer = D.

Q10 Text Solution:

In the case of BFS, if we draw a complete binary tree, then in Set A, we have level1+level2.



In DFS we have level 1 level 2 + level 3.

So, A - B = remaining element of level 2.

For example:-

In the given complete binary tree shown above, A – {A, B, C} and B – {A, B, D}. Here, A and B are sets containing the first 3 elements we get while performing BFS and DFS respectively.

In B, more than one sequence is possible, and we can choose any one of them.

So, |A – B| = |{A, B, C} – {A ,B, D}| = |{C}| = 1.

Q11 Text Solution:

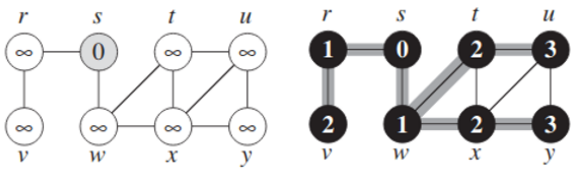
Without actually applying DFS, we can answer by just seeing options.

We know, In DFS, we go in depth first i.e., one node to another in depth first order.

Here, abfehg is not possible as we can not go from to directly.

Thus, option D is correct.

Q12 Text Solution:



Edges of G which are not in T are (t - x), (x - u), (u - y)

Option A, B, C satisfied

Answer is D.

d(u) – d(v) = 0 is possible when both u and v have an edge from a common node 't' and 't' is in the shortest path from 's' to 'u' and 'v'.

d(u) – d(v) = 1 is possible when 'v' and another node 't' are in the shortest path from 's' to 'u' and both 't' and 'v' are siblings- same distance from 's' to both 't' and 'v' causing (t – u) edge to be in BFS tree and not (v - u).

d(u) – d(v) = –1 is possible as explained above by interchanging 'u' and 'v'.

d(u) – d(v) = 2 is not possible. This is because on BFS traversal we either visit 'u' first or 'v'. Let's take 'u' first. Now, we put all neighbors of 'u' on queue. Since, 'v' is a neighbor and 'v' is not visited before as assumed, d(v) will become d(u) + 1. Similarly, for 'v' being visited first.

Android App | iOS App | PW Website